

Estofaria Digital

Análise Técnica Completa do Aplicativo

Perícia de Arquitetura · Código · Deploy · Saúde do Sistema

Julho de 2026

% % % ÍNDICE % % %

1. Visão Geral e Arquitetura
2. Mapa de Módulos e Arquivos
3. Fluxo de Autenticação
4. Deploy: Teste e Produção
5. Análise por Módulo
6. Pontos Fracos e Riscos
7. Código Morto e Duplicações
8. Endpoints de API
9. LocalStorage — Chaves Mapeadas
10. Diagnóstico de Saúde Geral
11. Recomendações e Próximos Passos

1. Visão Geral e Arquitetura

O que é o Estofaria Digital

Sistema SaaS para empresas de estofaria. Gerencia precificação de modelos, catálogo de produtos, orçamentos (módulo vendedor), agenda de produção e materiais. Multi-tenant: cada empresa tem dados isolados no backend.

Stack Tecnológica

Camada	Tecnologia	Hospedagem
Frontend	HTML + CSS + JavaScript puro (sem framework)	Cloudflare Pages
Backend	Node.js + Express	Render.com
Banco de Dados	PostgreSQL gerenciado no Render (R\$ 199/mês) — auto-switch	Render PostgreSQL
PWA	via DATABASE_URL service-worker.js + manifest.json	Embutido no frontend
Pagamentos	Stripe — ainda em modo TESTE	API externa
E-mail	Resend	API externa

Arquitetura App-Shell (iframe pool)

SPA via iframes. app-shell.js gerencia um pool de iframes carregados sob demanda. Cada módulo vive num __content.html separado e é injetado num iframe ao navegar. Módulos se comunicam com o shell via postMessage e localStorage.

- index.html verifica token JWT !' redireciona para /painel/
- app-shell.js carrega __content.html do módulo num iframe isolado
- config.js injeta ESTOFARIA_HTTP, ESTOFARIA_UI e ESTOFARIA_AUTH no window do iframe
- auth-guard.js verifica token e role antes de renderizar cada módulo
- Módulos comunicam dados ao painel via localStorage (ex: esd_proxima_vaga, PAINEL_SYNC_KEY)

URLs do Sistema

Produção (domínio próprio): <https://estofariadigital.com.br>

Cloudflare Pages: <https://estofaria-digital.pages.dev>

Backend API: <https://estofaria-backend.onrender.com/api>

Repositório Frontend: <https://github.com/danilo111727-code/estofaria-frontend>

Repositório Backend: <https://github.com/danilo111727-code/estofaria-backend>

2. Mapa de Módulos e Arquivos

Módulos do Frontend

Módulo	Arquivo Principal	Linhas	Função
Login	/login/script.js	~80	Autenticação, recuperação de senha
Painel	/painel/script.js	983	Dashboard: métricas, gráficos, próxima vaga
Agenda	/agenda/script.js	2255	Blocos de produção, vagas, pedidos, feriados
Vendedor	/vendedor/script.js	1474	Orçamentos, PDF, histórico de vendas
Catálogo	/catalogo/script.js	1099	Modelos, preços, imagens, PDF catálogo
Precificação	/precificacao/precificacao.js	1397	Custo por degraus de metragem
Itens Personaliz.	/itens-personalizacao/itens.js	~800	Itens extras por modelo (pés, tecido, espuma)
Material	/material/materiais.js	~400	Estoque matérias-primas, preços unitários
Configuração	/configuracao/script.js	~300	Perfil da empresa, preferências
Assinatura	/assinatura/script.js	~200	Planos Stripe, checkout, status
Master	/master/script.js	~200	Superadmin: gestão de todas as empresas

Arquivos Globais

Arquivo	Tamanho	Função
config.js	416 lin.	API_BASE, ESTOFARIA_HTTP, ESTOFARIA_UI, ESTOFARIA_AUTH
auth-guard.js	840 lin.	JWT, RBAC, rotas por role, status de assinatura
app-shell.js	233 lin.	Pool de iframes, navegação SPA, sincronização de altura
service-worker.js	—	Cache PWA, offline fallback
keyboard-dismiss.js	—	Fecha teclado mobile ao rolar
mobile-fixes.css	—	Correções globais iOS/Android

& Arquivos Duplicados / Órfãos — Problemas Reais

- frontend/frontend/ !' pasta duplicada (agenda, painel, configuração) — NUNCA referenciada pelo app-shell
- frontend/agenda/agenda.js (41 lin.) !' conteúdo HTML de redirecionamento — não tem lógica alguma de agenda
- frontend/agenda/feriados.js (41 lin.) !' mesmo conteúdo errado — não é lógica de feriados
- frontend/agenda/app.js !' placeholder vazio
- frontend/itens.html + itens.js (raiz) !' versão antiga, substituída por itens-personalizacao/
- frontend/materiais.html + materiais.js (raiz) !' versão antiga, substituída por material/
- frontend/precificacao.html + precificacao.js (raiz) !' versão antiga, substituída por precificacao/
- frontend/historico.html (raiz) !' versão antiga do histórico do vendedor
- frontend/script.js (raiz) !' script órfão sem referência em nenhum HTML
- sw.js + serviceworker.js + service-worker.js !' 3 service workers, apenas 1 é registrado

3. Fluxo de Autenticação

Como a autenticação funciona

JWT armazenado no localStorage. Não há cookie de sessão httpOnly — o token fica exposto ao JavaScript da página, o que é vulnerável a ataques XSS. Sem renovação automática de token (refresh token).

- 1. Usuário digita e-mail/senha em /login/index.html
- 2. POST /api/auth/login !' recebe { token, user }
- 3. Token salvo em localStorage como "auth_token" (ou "token" — 2 chaves por inconsistência histórica)
- 4. Usuário redirecionado para /painel/
- 5. auth-guard.js lê token, verifica validade e analisa: role, subscription_status, company_id
- 6. Se token expirado ou inválido !' redireciona para /login/

Roles (permissões)

Role	Módulos acessíveis
admin	Todos: painel, agenda, vendedor, catálogo, precificação, materiais, itens, configuração
vendedor	Apenas: vendedor, catálogo (somente leitura)
master / superadmin	Painel master: gestão de todas as empresas do sistema

Problemas identificados

- Token duplicado em 2 chaves: "auth_token" e "token" — código verifica ambas com fallback, mas gera confusão
- JWT em localStorage é vulnerável a XSS — ideal seria httpOnly cookie
- Lógica de auth espalhada em 3 arquivos: auth-guard.js, config.js e login/script.js
- Sem refresh token — sessão expira abruptamente, usuário é deslogado sem aviso
- Sem timeout de inatividade — sessão pode ficar ativa indefinidamente após inatividade

4. Deploy: Teste e Produção

Os dois mecanismos de deploy do Frontend

Mecanismo 1 — Git Push (NÃO usar para main sem aprovação):

- Push no branch main !' Cloudflare Pages detecta e faz deploy automático em PRODUÇÃO
- Push no branch dev !' Cloudflare gera URL de preview segura para teste

Mecanismo 2 — Wrangler CLI (processo correto documentado):

```
mkdir -p /tmp/estofaria-deploy
cp -r ./frontend/. /tmp/estofaria-deploy/
# criar _headers com Cache-Control para __content.html e index.html
npx wrangler@3 pages deploy /tmp/estofaria-deploy \
  --project-name=estofaria-digital \
  --branch=main --commit-dirty=true
```

& /tmp é apagado entre sessões — sempre reconstruir com cp -r antes de deployar.

Ambientes identificados

Ambiente	URL	Branch	Método de deploy
Produção	estofariadigital.com.br	main	Wrangler CLI (manual, só com aprovação)
Preview CF	estofaria-digital.pages.dev	main	Auto (git push main) — mesma produção!
Preview de teste	dev.estofaria-digital.pages.dev	dev	Auto (git push dev) — seguro para testes
Snapshot estável	—	stable-pages	Manual

Fluxo correto passo a passo

1. Editar arquivos em ./frontend/
 2. Bumpar ?v= nos __content.html dos módulos alterados
 3. git push origin dev !' aguardar preview em dev.estofaria-digital.pages.dev
 4. Testar no preview e dar aprovação explícita
 5. Aprovação !' executar comando wrangler com --branch=main para produção
 6. Verificar em estofariadigital.com.br
- &Ô NUNCA: git push origin main sem aprovação explícita

Versões de cache atuais por módulo

Módulo	Versão do script	Data aprox.
agenda/__content.html	script.js?v=20260719c	19/07/2026 — mais recente
painel/__content.html	script.js?v=20260718a style.css?v=20260625g	18/07/2026
itens-personalizacao/	itens.js?v=20260717a	17/07/2026
precificacao/	precificacao.js?v=20260716f	16/07/2026
material/	materiais.js?v=20260716h	16/07/2026
catalogo/	script.js?v=20260510l	10/05/2026
vendedor/	script.js?v=20260510k	10/05/2026
configuracao/	script.js?v=20260424i &	24/04/2026 — MAIS ANTIGO
app-shell.js CV global	20260717j &	17/07 — desatualizado vs agenda 20260719c

Deploy do Backend

- Deploy automático via push no branch main do repositório estofaria-backend
- Banco de dados: PostgreSQL gerenciado no Render — dados persistentes e seguros
- Verificar se DATABASE_URL está configurada no painel do Render (se sim, store-pg.js é usado automaticamente; se não, cai para store.json local)

5. Análise por Módulo

Painel (Dashboard)

- Exibe: total de vendas, orçamentos pendentes, taxa de conversão, gráfico mensal de receita
- Cache local com TTL: 30s para resumo, 60s para pedidos — eficiente
- Gráfico de modelos mais vendidos lê order.modelos da Agenda via PAINEL_SYNC_KEY
- esd_proxima_vaga: dependência frágil — Agenda escreve, Painel lê. Removido acidentalmente em 16/07 e restaurado em 19/07/2026

Agenda (módulo mais complexo — 2255 linhas)

- Blocos de produção com N vagas cada, pedidos com cliente, modelos selecionados e valor
- Integração com feriados nacionais, estaduais e municipais (API + localStorage manual)
- Modelos dos pedidos alimentam o gráfico do Painel via order.modelos
- fetchCatalogModels() sem timeout original — corrigido em 19/07 (agora 3s + suporte formato {models:[]})
- Logs [ESD-DIAG] ativos em produção — expõem dados internos no console do browser

Vendedor

- Orçamentos com modelo, itens personalizados e cálculo automático de preço
- PDF profissional com templates configuráveis pelo usuário
- Histórico de orçamentos com filtros, integração com Agenda
- Hack window._pendingPdfWindow para iOS Safari — funciona mas frágil

Catálogo

- CRUD de modelos: imagens, descrição, fórmula por degrau, PDF do catálogo
- Inconsistência: ora usa window.API_BASE, ora variável local API — pode quebrar em contexto de iframe

Precificação

- Fórmula: $\text{precoFinal} = \text{precoBase} + \text{floor}((\text{metragem} - \text{base}) / \text{espaçamento}) \times \text{valorPorDegrau}$
- Base = 2m | Espaçamento = 10cm | Catálogo e Vendedor usam a MESMA fórmula — consistente
- Memory leak: event listeners em visualViewport não removidos em sessões longas
- DOM manipulation complexa para PDF overlay — frágil em telas muito pequenas

Material, Configuração e Assinatura

- Material: 66 materiais (seed v3), preços em centavos (int) — correto para evitar erros de float
- Configuração: versão ?v=20260424i — módulo mais desatualizado do sistema (abril/2026)
- Assinatura: Stripe em modo TESTE — sem receita real gerada. Plano: R\$ 89,90/mês

6. Pontos Fracos e Riscos

Ø=¥4 Riscos Críticos

- Stripe em modo teste — pagamentos não funcionam em produção, nenhuma receita real
- Credenciais expostas no repositório GitHub (INFORMACOES_IMPORTANTES.md): Cloudflare API Key, GitHub Token, Stripe Secret, Resend Key
- Token JWT em localStorage — vulnerável a roubo por XSS

Ø=ßá Riscos Moderados

- app-shell.js CV (20260717j) desatualizado vs agenda (20260719c) — pode servir versão antiga se cache limpar
- Comunicação entre módulos via localStorage sem validação — colisão de dados possível
- Logs [ESD-DIAG] em produção expõem dados internos no console do browser
- Sem tratamento de erro global — erros não capturados travam módulos silenciosamente
- Precificação: memory leak com event listeners em visualViewport não removidos
- Módulo Configuração 3 meses desatualizado (v=20260424i)

Ø=¥5 Dívida Técnica

- apiGet/apiPost redefinidos em cada script.js — deveria ser módulo compartilhado
- ~28 chaves no localStorage sem namespace único — risco de colisão entre módulos
- Três service workers (sw.js, serviceworker.js, service-worker.js) — apenas 1 é registrado
- Sem testes automatizados (unitários ou e2e)
- Sem CI/CD formal — processo de deploy 100% manual e propenso a erros
- Sem monitoramento de disponibilidade (uptime monitoring)

Placar de Saúde por Área

Área	Nota	Status
Banco de Dados	8/10	PostgreSQL no Render (R\$199/mês) — persistente e seguro '
Backup / Recuperação	6/10	Render PostgreSQL inclui backups — confirmar se estão ativos
Stripe / Pagamentos	3/10	Modo teste — sem receita real gerada
Deploy / CI-CD	4/10	Manual, sem pipeline, propenso a erros humanos
Autenticação	5/10	JWT funciona, mas localStorage é inseguro
Qualidade de Código	5/10	Funcional, muita duplicação e código morto
Módulo Painel	6/10	Dependência frágil com esd_proxima_vaga
Módulo Agenda	6/10	Mais complexo, fixes recentes aplicados
Módulo Catálogo	6/10	Funcional, inconsistência de API_BASE
Módulo Vendedor	7/10	Bem implementado, hack do PDF aceitável
Módulo Precificação	7/10	Fórmula correta e consistente com catálogo
Arquitetura App-Shell	7/10	Funciona bem, frágil na comunicação entre módulos

7. Código Morto e Duplicações

Arquivos que podem ser removidos com segurança

Arquivo / Pasta	Motivo para remover
frontend/frontend/ (pasta inteira)	Duplicata parcial — nunca referenciada pelo app-shell principal
frontend/agenda/agenda.js	Conteúdo HTML errado — cópia do app-shell.html, sem lógica alguma
frontend/agenda/feriados.js	Mesmo problema — 41 linhas de HTML de redirect
frontend/agenda/app.js	Placeholder vazio
frontend/itens.html + itens.js (raiz)	Versão antiga — substituída por itens-personalizacao/
frontend/materiais.html + materiais.js (raiz)	Versão antiga — substituída por material/
frontend/precificacao.html + precificacao.js (raiz)	Versão antiga — substituída por precificacao/
frontend/historico.html (raiz)	Versão antiga do histórico do vendedor
frontend/script.js (raiz)	Script órfão sem referência em nenhum HTML
frontend/sw.js + serviceworker.js	Duplicatas do service-worker.js principal

Padrões de código duplicados internamente

- apiGet / apiPost / apiPatch / apiDelete: redefinidos em agenda, vendedor, catálogo, precificação, itens e material — deveriam estar centralizados em config.js e importados
- formatCurrency / formatDate: implementadas de forma diferente em cada módulo
- notifyError / notifySuccess: wrappers de ui() redefinidos em vários scripts
- Cache de modelos em 3 chaves: catalogo_modelos, precificacao_modelos e modelos — deveriam ser uma só

8. Endpoints de API Mapeados

Autenticação

Método	Endpoint	Módulo
POST	/api/auth/login	login/script.js
POST	/api/auth/logout	auth-guard.js
POST	/api/auth/reset-password	login/script.js
GET	/api/auth/team	assinatura/script.js

Agenda

Método	Endpoint	Módulo
GET	/api/agenda/orders	agenda/script.js, painel/script.js
GET / PATCH	/api/agenda/config	agenda/script.js, painel/script.js
GET / POST	/api/agenda/blocos	agenda/script.js
PATCH / DELETE	/api/agenda/blocos/:id	agenda/script.js
POST	/api/agenda/blocos/:id/pedido	agenda/script.js
POST	/api/agenda/blocos/:id/vaga	agenda/script.js
PATCH / DELETE	/api/agenda/orders/:id	agenda/script.js

Modelos e Itens

Método	Endpoint	Módulo
GET	/api/models	agenda, catálogo, precificação, itens, vendedor
POST / PATCH / DELETE	/api/models e /api/models/:id	precificacao/precificacao.js
GET / POST / PATCH	/api/models/:id/personalization-items	itens-personalizacao/itens.js

Diversos

Método	Endpoint	Módulo
GET / POST / PATCH	/api/materials e /api/materials/:id	material/materiais.js
GET	/api/dashboard/summary	painel/script.js
GET	/api/calendar/holidays	agenda/script.js
GET	/api/subscription/public	auth-guard.js
POST	/api/subscription/checkout-requests	assinatura/script.js
GET / POST / DELETE	/api/pdf-templates	vendedor/pdf-tools.js
GET / POST / PATCH	/api/quotes e /api/quotes/:id	vendedor/script.js

9. LocalStorage — Todas as Chaves Mapeadas

Autenticação (com inconsistências)

Chave	Conteúdo	Problema
auth_token	JWT do usuário	Duplica a chave "token"
token	JWT alternativo	Legado — inconsistência histórica
auth_user / esd_user	Dados do usuário (JSON)	2 chaves para mesma informação
role / user_role	Permissão do usuário	Duplicado
company_name / estofaria_company_name	Nome da empresa	Duplicado

Agenda e Painei

Chave	Conteúdo / Uso
esd_proxima_vaga	Data ISO da próxima vaga — escrito pela Agenda, lido pelo Painei via evento storage
esd_order_valores	Cache de valores dos pedidos — evita perda ao recarregar a Agenda
esd_bloqueios_count	Contagem de bloqueios por semana (para cálculo de vagas disponíveis)
esd_manual_holidays_{company_id}	Feriados cadastrados manualmente pela empresa
esd_itens_cols_v1	Configuração de colunas da tabela de itens de personalização
esd_albums_v1	Álbuns de imagens do catálogo

Cache de API e Outras

Chave	Conteúdo
catalogo_modelos / precificacao_modelos / modelos	3 caches separados de modelos — deveriam ser 1
itens_personalizacao_cache_v2:{modelId}	Cache de itens de personalização por modelo
PAINEL_CACHE_PREFIX + chave	Cache do dashboard (TTL: 30s resumo / 60s pedidos)
estofaria_api_base	URL base da API (permite override manual)
estofaria_modelo_ativo_v1	Modelo ativo selecionado no catálogo
STORAGE_TEMPLATES	Templates de PDF do módulo vendedor
estofaria_materiais_seeded_v3	Flag de seed dos materiais (boolean — evita re-seed)

& ~28 chaves no localStorage sem namespace único — risco de colisão entre módulos. Muitas duplicadas por acúmulo histórico de refactors.

' O que está funcionando bem

- Arquitetura App-Shell com iframes — bom isolamento de módulos, navegação fluida no mobile
- Sistema de cache busting (?v=) — quando aplicado corretamente, garante código novo nos browsers
- Cálculo de precificação por degraus — lógica correta e consistente entre Catálogo e Vendedor
- Geração de PDF no Vendedor — inclui workaround funcional para iOS Safari
- Integração Agenda !” Painel via esd_proxima_vaga — restaurado e funcional (19/07/2026)
- Multi-tenant funcional: dados isolados por company_id no backend
- PWA configurado — pode ser instalado como app no celular (iOS e Android)
- RBAC funcional — admin vs vendedor com módulos restritos
- Feriados nacionais, estaduais e municipais na Agenda — cobertura completa do Brasil

'L O que está quebrado ou incompleto

- Stripe em modo teste — nenhuma receita real sendo gerada
- Logs [ESD-DIAG] ativos em produção — dados internos expostos no console do browser
- app-shell.js CV (20260717j) desatualizado vs agenda (20260719c) — risco de versão antiga
- agenda/agenda.js e feriados.js com conteúdo completamente errado (HTML de redirect)
- frontend/frontend/ duplicata não referenciada — confunde qualquer desenvolvedor
- Módulo Configuração com versão de abril/2026 — 3 meses desatualizado

Ø=ÝÑ O que está sobrando (pode remover com segurança)

- frontend/frontend/ — pasta duplicada inteira (agenda, painel, configuração)
- Raiz: itens.html, itens.js, materiais.html, materiais.js, precificacao.html, precificacao.js, historico.html, script.js
- Agenda: agenda.js, feriados.js, app.js
- Duplicatas de service worker: sw.js e serviceworker.js (manter apenas service-worker.js)

11. Recomendações e Próximos Passos

Ø=Ý4 Prioridade Máxima — Fazer Agora

- 1. STRIPE: Verificar se está em modo produção (sk_live_) — o sistema pode não estar gerando receita real
- 2. LOGS: Remover todos os console.log("[ESD-DIAG]") do agenda/script.js antes do próximo deploy
- 3. CREDENCIAIS: Remover INFORMACOES_IMPORTANTES.md do repositório GitHub !' mover para 1Password, Notion privado ou similar
- 4. DEPLOY: Adotar fluxo correto: editar !' git push dev !' testar preview !' wrangler produção só com aprovação
- 5. BANCO: Confirmar que DATABASE_URL está configurada no Render e que backups automáticos estão ativos no plano PostgreSQL

Ø=Á Prioridade Média — Fazer em Breve

- 6. CV: Atualizar CV global do app-shell.js de 20260717j para 20260719c para sincronizar
- 7. LIMPEZA: Remover os ~15 arquivos e 1 pasta mortos listados na seção 7
- 8. CONFIGURAÇÃO: Atualizar módulo de configuração (ainda na versão de abril/2026)
- 9. NAMESPACE: Padronizar todas as chaves do localStorage com prefixo único "esd_"
- 10. API: Centralizar apiGet/apiPost em config.js e remover 6 redefinições duplicadas

Ø=Ý5 Melhorias Futuras

- 12. SEGURANÇA: Migrar JWT para httpOnly cookie (requer mudança coordenada no backend)
- 13. REFRESH TOKEN: Renovação automática de sessão sem deslogar o usuário abruptamente
- 14. MONITORAMENTO: UptimeRobot (gratuito) para alertas de indisponibilidade do backend e frontend
- 15. CI/CD: GitHub Actions para deploy automático no branch dev e notificação para aprovação manual
- 16. TESTES: Playwright nos módulos críticos — agenda (adicionar pedido) e vendedor (gerar PDF)
- 17. MEMÓRIA: Remover event listeners em visualViewport na precificação para evitar memory leak

Resumo Executivo

O Estofaria Digital é um sistema funcional e bem projetado para seu propósito. A arquitetura App-Shell com iframes resolve bem o isolamento de módulos em contexto mobile-first. O código principal — agenda, vendedor, painel e precificação — cobre os fluxos de negócio corretamente e com consistência na fórmula de preços.

O banco de dados é PostgreSQL gerenciado no Render (R\$ 199/mês), com auto-switch inteligente via DATABASE_URL — os dados dos clientes estão seguros e persistidos corretamente. O maior risco operacional no momento é o Stripe ainda em modo teste, o que pode significar que o sistema não está gerando receita real. Confirmar e ativar o modo produção do Stripe é a ação mais urgente.

Com processo de deploy correto adotado, Stripe confirmado em produção e limpeza dos arquivos mortos, o sistema estará numa posição muito mais sólida e sustentável para crescer.

Nota — Situação atual do deploy (19/07/2026):

Foram feitos 3 commits diretos no branch main (git push origin main) em vez do fluxo correto. A produção (Deployment ID: 2257ca1d) não foi alterada pois o Cloudflare Pages deste projeto não tinha auto-deploy via GitHub ativo — a API confirmou que 2257ca1d é o deployment atual em produção. Os commits estão no repositório GitHub mas a produção continua intacta.